

5.6 — Preserve Information Provenance and Handle Uncertainty in Synthesis

Concept

When an agent synthesizes information from multiple sources, it must do more than produce a coherent summary. It must preserve the audit trail — which claim came from which source — flag conflicts rather than silently resolving them, respect temporal relationships between data, and represent uncertainty honestly in its output. Failing to do any of these produces outputs that appear authoritative but cannot be trusted or debugged.

Information Provenance

Provenance is the documented origin of a claim. In a retrieval-augmented or multi-source pipeline, every synthesized statement should be traceable to a specific source: a document ID, a URL, a database row, or a paragraph offset. Without this tracing, downstream consumers cannot verify the claim, cannot re-fetch the source for more context, and cannot audit the agent's reasoning.

Design your data structures to carry provenance from the moment of ingestion. A retrieved chunk should never lose its origin metadata as it flows through embedding, retrieval, and synthesis stages.

Claim-Source Mappings

A claim-source mapping binds a discrete assertion to the source that supports it. The minimum useful mapping records: the claim text, the source identifier, the location within the source (page, paragraph, or character offset), and the retrieval timestamp. Richer mappings also carry a confidence score and the verbatim excerpt that supports the claim.

In a structured output context, this means your synthesis schema must include a `sources` array on every claim, not just on the overall response. "The company was founded in 2012" is a claim; it needs its own source annotation.

Conflict Annotation

When two sources disagree, the agent must flag the conflict rather than picking a winner silently. Silent resolution is a form of hallucination: the agent presents a single answer with false certainty. Explicit conflict annotation surfaces the disagreement to the caller, who can apply domain knowledge, fetch a tiebreaker source, or present both versions to the user.

The correct pattern is: detect the conflict during synthesis, emit both claims with their respective sources, and attach a `conflict` flag. Do not average, do not prefer the more recent source automatically, and do not omit the minority claim.

Temporal Data Handling

Information has timestamps. A price from last quarter, a regulation from 2021, and a real-time API response live in different temporal contexts. Your synthesis layer must track `retrievedAt` and `publishedAt` for each source, compare timestamps when claims conflict, and surface temporal distance as a factor — not as a deterministic override.

A newer source does not automatically win. A peer-reviewed paper from 2019 may be more reliable than a blog post from yesterday. Timestamp comparison narrows the candidate set; domain rules and confidence scores make the final call.

Handling Uncertainty in Synthesis

When the evidence is ambiguous, partial, or contradictory, the output must reflect that. Never present an uncertain synthesis as a definitive answer. Use a `confidence` field (a normalized 0–1 float is conventional) and a `reasoning` field that explains what drove the score down. A confidence of 0.45 means "the evidence is split"; a confidence of 0.9 means "three independent sources agree."

Uncertainty propagates: a claim derived from two conflicting sources inherits a lower confidence than a claim with three concordant sources, regardless of how well-written the synthesis prose is.

Code Example

Provenance-Aware Synthesis Schema

```
interface SourceRef {
  sourceId: string;      // document ID or URL
  location: string;      // "page 4, paragraph 2" or character offset range
  excerpt: string;       // verbatim supporting text
  publishedAt: string;    // ISO 8601
  retrievedAt: string;    // ISO 8601
}

interface SynthesizedClaim {
  claim: string;
  confidence: number;     // 0.0 – 1.0
  sources: SourceRef[];
  conflict: boolean;      // true if sources disagree on this claim
  conflictSummary?: string; // required when conflict === true
}

interface SynthesisResult {
  summary: string;
  claims: SynthesizedClaim[];
  synthesizedAt: string;
}
```

Conflict Detection During Synthesis

```
function detectConflict(claims: SynthesizedClaim[]): SynthesizedClaim[] {
  return claims.map((claim) => {
    const excerpts = claim.sources.map((s) => s.excerpt.toLowerCase().trim());
    const allAgree = excerpts.every((e) => e === excerpts[0]);

    if (!allAgree) {
      const summary = claim.sources
        .map((s) => `${s.sourceId} "${s.excerpt}"`)
        .join(" vs. ");

      return { ...claim, conflict: true, conflictSummary: summary };
    }

    return { ...claim, conflict: false };
  });
}
```

Prompting Claude to Emit Provenance-Aware Output

```
import Anthropic from "@anthropic-ai/sdk";

const client = new Anthropic();

const sources = [
  {
    id: "report-2023",
    text: "The market grew 12% in 2023.",
    publishedAt: "2023-11-01",
  },
  {
    id: "report-2024",
    text: "The market grew 9% in 2023.",
    publishedAt: "2024-02-15",
  },
];

const sourceBlock = sources
  .map((s) => `${s.id} (published ${s.publishedAt}): ${s.text}`)
  .join("\n");

const response = await client.messages.create({
  model: "claude-opus-4-5",
  max_tokens: 512,
  tools: [
```

```

{
  name: "emit_synthesis",
  description: "Emit a structured synthesis with claim-source mappings.",
  input_schema: {
    type: "object",
    properties: {
      claims: {
        type: "array",
        items: {
          type: "object",
          properties: {
            claim: { type: "string" },
            confidence: { type: "number" },
            sourceIds: { type: "array", items: { type: "string" } },
            conflict: { type: "boolean" },
            conflictSummary: { type: "string" },
          },
          required: ["claim", "confidence", "sourceIds", "conflict"],
        },
      },
      required: ["claims"],
    },
  },
  tool_choice: { type: "any" },
  messages: [
    {
      role: "user",
      content: `Synthesize the following sources. Flag any conflicts. Do not resolve conflicts
    },
  ],
});

// Extract tool_use block
const toolUse = response.content.find((b) => b.type === "tool_use");
if (toolUse && toolUse.type === "tool_use") {
  const synthesis = toolUse.input as { claims: SynthesizedClaim[] };
  const annotated = detectConflict(synthesis.claims);
  console.log(JSON.stringify(annotated, null, 2));
}

```

Temporal Tie-Breaking (Advisory Only)

```

function rankByRecency(sources: SourceRef[]): SourceRef[] {
  return [...sources].sort(

```

```

(a, b) =>
  new Date(b.publishedAt).getTime() - new Date(a.publishedAt).getTime()
);
}

function temporalConfidencePenalty(sources: SourceRef[]): number {
  if (sources.length < 2) return 0;
  const sorted = rankByRecency(sources);
  const newestMs = new Date(sorted[0].publishedAt).getTime();
  const oldestMs = new Date(sorted[sorted.length - 1].publishedAt).getTime();
  const daysDelta = (newestMs - oldestMs) / (1000 * 60 * 60 * 24);
  // Penalise confidence by up to 0.2 when sources span more than a year
  return Math.min(0.2, daysDelta / 1825);
}

```

Common Mistakes

Merging conflicting claims into a single sentence. Writing "the market grew approximately 10–12%" when two sources give different numbers hides the disagreement. Always emit both values with their source IDs and set `conflict: true`.

Dropping source metadata during retrieval. Embedding pipelines often discard document IDs and timestamps to save tokens. Once metadata is gone it cannot be reconstructed. Carry `sourceId`, `publishedAt`, and `location` through every pipeline stage.

Treating recency as authority. Automatically picking the most recent source ignores source quality, domain context, and publication intent. Use timestamp as one signal, not the decisive one.

Presenting low-confidence claims without qualification. A claim with confidence 0.3 must not be phrased as a statement of fact. Either surface the uncertainty to the caller via the `confidence` field, or add hedging language in the `claim` text itself.

Omitting `conflictSummary` when `conflict` is true. Flagging a conflict without explaining it forces the caller to re-derive the disagreement from raw sources. Always include a brief human-readable description of what the sources say and why they disagree.

Using a single top-level source citation for the whole response. Citing "sources: [A, B, C]" at the response level does not tell the caller which source supports which claim. Provenance must be per-claim.

References

- Anthropic docs: [Tool use — structured output](#)
- Anthropic docs: [Retrieval-augmented generation patterns](#)
- Pattern: claim-source mapping (information retrieval literature, Hearst 1992; modern RAG attribution)
- Exam objectives: Section 5 — Reliability, Safety, and Output Quality

